

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: Jacobi 方法、QR 算法与 eig 函数对比

学院: 计算机与软件学院

专业: 人工智能卓越班

指导教师: 吴晓群、卯丙

报告人: 邓瑞霖 学号: 2024150040 班级: 01

实验时间: 2026 年 4 月 22 日 ~ 2026 年 5 月 6 日

实验报告提交时间: 2026 年 4 月 26 日星期日

教务部制

实验目的与要求:

- 1.掌握 Jacobi 法与 QR 方法的的基本思想和算法步骤
- 2.通过编程实现方阵 A 的所有特征值及其对应特征向量的计算算法, 验证理论结果
- 3.比较 Jacobi 法与 QR 方法的收敛性
- 4.比较 Jacobi 法、QR 方法与 MATLAB 中 eig 函数的求解精度

基本原理与算法 (详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤)

1. Jacobi 方法

1) 任意的实对称矩阵 A 可通过正交相似变化成对角阵, 即存在征集哦啊矩阵 Q, 使得

$$Q^T A Q = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$$

其中 $\lambda_i (i = 1, 2, \dots, n)$ 是 A 的特征值, Q 中各列即为相应的特征向量

2) 在正交相似变换下, 矩阵元素的平方和不变

设 $A = (a_{ij})_{n \times n}$, Q 为正交矩阵, 记 $B = Q^T A Q = (b_{ij})_{n \times n}$, 则 $\sum_{i,j=1}^n a_{ij}^2 = \sum_{i,j=1}^n b_{ij}^2$,

$$\text{即 } \|RA\|_F^2 = \|AR\|_F^2 = \|RAR^T\|_F^2 = \|A\|_F^2$$

通过一次正交变换, 将 A 中一对非零的非对角元素化成零, 并且使得非对角元素的平方和减少。反复进行上述过程, 使变换后的矩阵的非对角元素的平方和趋于零, 从而使该矩阵近似为对角矩阵, 得到全部特征值和特征向量

3) 平面上的一条二次曲线 $a_{11}x_1^2 + 2a_{12}x_1x_2 + a_{22}x_2^2 = 1$

矩阵形式为: $\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 1$ 其中 $a_{12} = a_{21}$

可以通过坐标轴的旋转 $\begin{cases} x_1 = y_1 \cos \theta - y_2 \sin \theta \\ x_2 = y_1 \sin \theta + y_2 \cos \theta \end{cases}$

$$\text{即 } \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

化为标准化形式: $\lambda_1 y_1^2 + \lambda_2 y_2^2 = 1$

$$\text{即 } \begin{bmatrix} y_1 & y_2 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = 1$$

过程如下:

$$\begin{aligned} [y_1 \ y_2] \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \\ = [y_1 \ y_2] \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = 1 \end{aligned}$$

其中

$$\begin{aligned} b_{11} &= a_{11}\cos^2\theta + a_{22}\sin^2\theta + a_{21}\sin 2\theta \\ b_{22} &= a_{11}\sin^2\theta + a_{22}\cos^2\theta - a_{21}\sin 2\theta \\ b_{12} &= b_{21} = \frac{1}{2}(a_{22} - a_{11})\sin 2\theta + a_{21}\cos 2\theta \end{aligned}$$

若取 θ 使得 $b_{12} = b_{21} = 0$,

则上式可以写成 $[y_1 \ y_2] \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = 1$ 或 $R \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} R^T = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$, 其

中 $R = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$, R 是一个正交矩阵。称正交矩阵把对称矩阵 A 变成为对角

矩阵, λ_1, λ_2 是矩阵 A 的特征值, 正交矩阵 R 的两个列向量分别为对应两个特征值的单位特征向量

4) Givens 变换 (矩阵平面旋转变换)

设 A 为 n 阶实对称矩阵, 考虑矩阵:

$$V_{ij}(\varphi) = \begin{bmatrix} 1 & & & & & & & \\ & \ddots & & & & & & \\ & & 1 & & & & & \\ & & & \cos\varphi & & & \sin\varphi & \\ & & & & 1 & & & \\ & & & & & \ddots & & \\ & & & & & & 1 & \\ & & & -\sin\varphi & & & \cos\varphi & \\ & & & & & & & 1 & \\ & & & & & & & & \ddots & \\ & & & & & & & & & 1 \end{bmatrix} \quad \text{称为 Givens 矩阵}$$

或平面旋转变换矩阵

$V_{ij}(\varphi)$ 是正交矩阵, 若记 $A^{(1)} = V_{ij}^T A V_{ij} = (a_{ij}^{(1)})$

$$\text{则有} \begin{cases} a_{ii}^{(1)} = a_{ii} \cos^2 \varphi + a_{jj} \sin^2 \varphi + a_{ij} \sin 2 \varphi \\ a_{jj}^{(1)} = a_{ii} \sin^2 \varphi + a_{jj} \cos^2 \varphi - a_{ij} \sin 2 \varphi \\ a_{ik}^{(1)} = a_{ki}^{(1)} = a_{ik} \cos \varphi + a_{jk} \sin \varphi & (k \neq i, j) \\ a_{jk}^{(1)} = a_{kj}^{(1)} = -a_{ik} \sin \varphi + a_{jk} \cos \varphi \\ a_{kl}^{(1)} = a_{lk}^{(1)} = a_{kl} & (k, l \neq i, j) \\ a_{ij}^{(1)} = a_{ji}^{(1)} = \frac{1}{2}(a_{jj} - a_{ii}) \sin 2 \varphi + a_{ij} \cos 2 \varphi \end{cases}$$

若 $a_{ij} \neq 0$, 取 φ 使得 $\tan 2\varphi = 2a_{ij}/(a_{jj} - a_{ii})$ ($|\varphi| < \pi/4$) 则有 $a_{ij}^{(1)} = a_{ji}^{(1)} = 0$ 得到

一个使 A 中非零的非对角元素 $a_{ij} = a_{ji}$ 变成零的正交相似变换。

对 $A^{(1)}$ 重复上述过程 $\rightarrow A^{(2)} \rightarrow \dots$, 得到一个矩阵序列 $\{A^{(k)}\}$

1.1 Jacobi 方法

通过一系列旋转相似变换将 A 变成 $A^{(k+1)}$ ，求的 A 的全部特征值和特征向量的方法称为 Jacobi 方法。若在对 A 做相似变换的过程中，每一步都选绝对值最大的非对角元素

$a_{ij}^{(k)}$ ，以此确定旋转矩阵的方法称为古典 Jacobi 方法

设 $A^{(0)} = A$ 由 $A^{(0)}$ 出发对其左 Givens 变换 $G_0(p, q)$ ，一般地，有

$$A^{(k)} = G_k^T A^{(k-1)} G_k$$

其中 $G_k(p, q) = \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & \cos\theta & \dots & \sin\theta \\ & & & & 1 & \\ & & \vdots & & \ddots & \vdots \\ & & & -\sin\theta & \dots & \cos\theta \\ & & & & & & 1 & & \\ & & & & & & & \ddots & \\ & & & & & & & & 1 \end{pmatrix}$ 为 Givens 矩阵，

为使 $A^{(k)}$ 趋向一解矩阵 ($k \rightarrow \infty$)，可以这样确定 $G_k(p, q)$ ：

对于任意 $k > 0$ ， p, q 由 $A^{(k-1)}$ 中非对角元素 $a_{ij}^{(k-1)} (i \neq j)$ 按模最大值的脚标决定，令

$a_{pq}^{(k)} = 0$ 来确定 θ

A 是实对称矩阵， $A^{(k)}$ 与 $A^{(k-1)}$ 只有 p, q 两行（列）不同，它们之间的关系是

$$\begin{cases} a_{pi}^{(k)} = a_{ip}^{(k)} = a_{ip}^{(k-1)} \cos\theta + a_{iq}^{(k-1)} \sin\theta & (i \neq p, q), \\ a_{qi}^{(k)} = a_{iq}^{(k)} = -a_{ip}^{(k-1)} \sin\theta + a_{iq}^{(k-1)} \cos\theta & (i \neq p, q), \\ a_{pp}^{(k)} = a_{pp}^{(k-1)} \cos^2\theta + 2a_{pq}^{(k-1)} \sin\theta \cos\theta + a_{qq}^{(k-1)} \sin^2\theta, \\ a_{qq}^{(k)} = a_{pp}^{(k-1)} \sin^2\theta - 2a_{pq}^{(k-1)} \sin\theta \cos\theta + a_{qq}^{(k-1)} \cos^2\theta, \\ a_{pq}^{(k)} = a_{qp}^{(k)} = (a_{pp}^{(k-1)} - a_{qq}^{(k-1)}) \sin\theta \cos\theta + a_{pq}^{(k-1)} (\cos^2\theta - \sin^2\theta). \end{cases}$$

令 $a_{pq}^{(k)} = a_{qp}^{(k)} = 0$ ，解 θ 满足的条件

$$\tan 2\theta = \frac{2a_{pq}^{(k-1)}}{a_{qq}^{(k-1)} - a_{pp}^{(k-1)}}$$

通常取

$$|\theta| \leq \frac{\pi}{4}$$

若 $a_{pp}^{(k-1)} = a_{qq}^{(k-1)}$ ，则取

$$\theta = \begin{cases} -\pi/4 & a_{pq}^{(k-1)} < 0 \\ \pi/4 & a_{pq}^{(k-1)} > 0 \end{cases}$$

实际中不用计算 θ ，只需计算 $\sin\theta, \cos\theta$ ，令

$$y = |a_{qq}^{(k-1)} - a_{pp}^{(k-1)}|, x = \text{sign}(a_{qq}^{(k-1)} - a_{pp}^{(k-1)})2a_{pq}^{(k-1)}$$

则

$$\tan 2\theta = \frac{x}{y}$$

$$\cos\theta = \left(\frac{1}{2} \left(1 + \frac{y}{\sqrt{x^2 + y^2}} \right) \right)^{1/2}$$

$$\sin\theta = \frac{x}{2\cos\theta\sqrt{x^2 + y^2}}$$

1.2 算法步骤

1) 置 $k \leftarrow 0, A^{(k)} \leftarrow A$ ，误差限 $\epsilon (\epsilon > 0)$, $Q^{(k)} \leftarrow I$

2) 求整数 i, j ，使 $|a_{ij}^{(k)}| = \max_{1 \leq l, m \leq n, l \neq m} |a_{lm}^{(k)}|$

3) 计算 $V_{ij}(\varphi): a = \cot(2\varphi) = [a_{jj}^{(k)} - a_{ii}^{(k)}]/[2a_{ij}^{(k)}], b = \tan\varphi =$

$$\text{sign}(a) \cdot (\sqrt{a^2 + 1} - |a|), c = \cos\varphi = 1/\sqrt{1 + b^2}, d = \sin\varphi = bc$$

4) 置 $k \leftarrow k + 1$

5) 计算 $A^{(k)}$:

$$\begin{cases} a_{ii}^{(k)} = c^2 a_{ii}^{(k-1)} + d^2 a_{jj}^{(k-1)} - 2cd a_{ij}^{(k-1)} \\ a_{jj}^{(k)} = d^2 a_{ii}^{(k-1)} + c^2 a_{jj}^{(k-1)} + 2cd a_{ij}^{(k-1)} \\ a_{il}^{(k)} = a_{li}^{(k)} = c a_{il}^{(k-1)} - d a_{jl}^{(k-1)} & (l \neq i, j) \\ a_{jl}^{(k)} = a_{lj}^{(k)} = d a_{il}^{(k-1)} + c a_{jl}^{(k-1)} & (l \neq i, j) \\ a_{lm}^{(k)} = a_{ml}^{(k)} = a_{lm}^{(k-1)} & (l, m \neq i, j) \\ a_{ij}^{(k)} = a_{ji}^{(k)} = 0 \end{cases}$$

6) 计算 $Q^{(k)} = Q^{(k-1)} V_{ij}(\varphi)$

7) 计算 $E[A^{(k)}] = \sum_{l \neq m} [a_{lm}^{(k)}]^2$

8) 若 $E[A^{(k)}] < \epsilon$ ，则输出特征值的近似值 $a_{11}^{(k)}, a_{22}^{(k)}, \dots, a_{nn}^{(k)}$ 及 $Q \leftarrow Q^{(k)}$ (Q 的列向

量即相应特征值的特征向量的近似)；否则，返回步骤二

9) 停机

1.3 收敛性

设 A 是实对称矩阵，则 Jacobi 方法产生的序列 $\{A^{(k)}\}$ 的非对角元素收敛于零，即 $A^{(k)}$ 趋于对角矩阵 ($k \rightarrow \infty$)

2. QR 分解

设 $A \in \mathbb{R}^{n \times n}$ ，则 A 可以分解为

$$A = QR$$

其中 Q 为正交矩阵， R 为上三角矩阵。规定 R 的对角线元素是正实数时，分解是唯一的

2.1 Household 算法

2.1.1 Household 变换

设 $\omega \in \mathbb{R}^n$ ，且 $\|\omega\|_2 = \sqrt{\omega^T \omega} = 1$ ，称形如

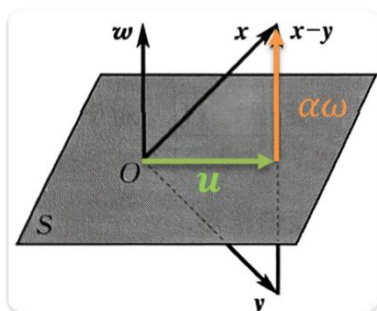
$$H = I - 2\omega\omega^T$$

的实对称矩阵 H 为 Household 变换，或称镜像变换、Household 矩阵、反射矩阵

2.1.2 Householder 变换的性质

称 H 是 Household 变换，则

- 1) H 是实对称矩阵的正交变换，即 $H^T = H, H^T H = I, H^{-1} = H^T = H$
- 2) H 仅有两个互异特征值 -1 和 1，其中 -1 为单重特征值，相应的特征子空间为 $\text{span}\{\omega\}$ ，1 是 H 的 $n-1$ 重特征值，与其相对应的特征子空间为 $(\text{span}\{\omega\})^\perp$ ，即 $\text{span}\{\omega\}$ 的正交补。
- 3) $\det(H) = -1$
- 4) 对 $x \in \mathbb{R}^n$ ， x 都可以表示为 $x = u + \alpha\omega$ ，其中 $u \in (\text{span}\{\omega\})^\perp, \alpha \in \mathbb{R}$ ，于是，有 $Hx = H(u + \alpha\omega) = Hu + \alpha H\omega = u - \alpha\omega$ ，且 $\|x\|_2 = \|Hx\|_2$ ，即 H 是关于超平面 $\omega^T v = 0 (v \in \mathbb{R}^n)$ (的任意向量) 的反射变换



2.1.3 Hessenberg 矩阵

设矩阵 $B \in \mathbb{R}^{n \times n}$, 如果当 $i > j + 1$ 时, $b_{ij} = 0$, 即

$$B = \begin{pmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ & \ddots & \ddots & \vdots \\ & & b_{n,n-1} & b_{nn} \end{pmatrix}$$

则称矩阵 B 为上 Hessenberg 矩阵, 设

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \dots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} = \begin{pmatrix} a_{11} & A_{12}^{(1)} \\ c_1 & A_{22}^{(1)} \end{pmatrix}$$

其中 $c_1 = (a_{21}, a_{31}, \dots, a_{n1})^T \in \mathbb{R}^{n-1}$, 设 $c_1 \neq 0$, 于是选反射矩阵 $R_1 = I -$

$\beta_1^{-1} u_1 u_1^T$ 使 $R_1 = \sigma_1 e_1$, 其中

$$\begin{cases} \sigma_1 = \text{sgn}(a_{21}) \left(\sum_{i=1}^n a_{i1}^2 \right)^{1/2} \\ u_1 = c_1 + \sigma_1 e_1 \\ \beta_1 = \sigma_1 (\sigma_1 + a_{21}) \end{cases}$$

令 $U_1 = \begin{pmatrix} 1 & \\ & R_1 \end{pmatrix}$, 则

$$\begin{aligned} A^{(2)} &= U_1 A^{(1)} U_1 = \begin{pmatrix} a_{11} & A_{12}^{(1)} R_1 \\ R_1 c_1 & R_1 A_{22}^{(1)} R_1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12}^{(2)} & \dots & a_{1n}^{(2)} \\ -\sigma_1 & a_{22}^{(2)} & \dots & a_{2n}^{(2)} \\ \vdots & \vdots & \dots & \vdots \\ 0 & a_{n2}^{(2)} & \dots & a_{nn}^{(2)} \end{pmatrix} \\ &= \begin{pmatrix} A_{11}^{(2)} & A_{12}^{(2)} \\ 0 c_2 & A_{22}^{(2)} \end{pmatrix} \end{aligned}$$

其中 $c_2 = (a_{32}^{(2)}, a_{42}^{(2)}, \dots, a_{n2}^{(2)})^T \in \mathbb{R}^{(n-2)}$, $A_{22}^{(2)} \in \mathbb{R}^{(n-2) \times (n-2)}$

第 k 步优化: 设对 A 已完成第一步, ..., 第 k-1 步正相似变换, 即有

$$A^{(k)} = U_{k-1} A^{(k-1)} U_{k-1} = \begin{pmatrix} A_{11}^{(k)} & A_{12}^{(k)} \\ 0 c_k & A_{22}^{(k)} \end{pmatrix}$$

其中 $c_k = (a_{k+1,k}^{(k)})^T \in \mathbb{R}^{n-k}$, $A_{11}^{(k)}$ 为 k 阶 Hessenberg 矩阵, $A_{22}^{(k)} \in \mathbb{R}^{(n-k) \times (n-k)}$

设 $c_k \neq 0$, 可选择初等反射矩阵 R_k , 使得 $R_k c_k = -\sigma_k e_1$, 其中 R_k 的计算公式为

$$\begin{cases} \sigma_k = \text{sgn}(a_{k+1,k}^{(k)}) \left(\sum_{i=k+1}^n (a_{ik}^{(k)})^2 \right)^{1/2} \\ u_k = c_k + \sigma_k e_k \\ \beta_k = \sigma_k (a_{k+1}^{(k)} + \sigma_k) \\ R_k = I - \beta_k^{-1} u_k u_k^T \end{cases}$$

令 $U_k = \begin{pmatrix} 1 & \\ & R_k \end{pmatrix}$, 则

$$A^{(2)} = U_k A^{(k)} U_k = \begin{pmatrix} A_{11}^{(k+1)} & A_{12}^{(k)} \\ 0 R_k c_k & R_k A_{22}^{(k)} R_k \end{pmatrix} = \begin{pmatrix} A_{11}^{(k+1)} & A_{12}^{(k+1)} \\ 0 c_{k+1} & A_{22}^{(k+1)} \end{pmatrix}$$

重复上述过程, 则有

$$U_{n-2} \dots U_2 U_1 A U_1 U_2 \dots U_{n-2} = \begin{pmatrix} a_{11} & & & \\ -\sigma_1 & a_{22}^{(2)} & & \\ & \ddots & \ddots & \\ & & -\sigma_{n-1} & a_{nn}^{(n-1)} \end{pmatrix} = A^{(n-1)}$$

2.2 利用 Household 矩阵左乘 A 来将 A 约化为上三角阵 R

记 $e_k = (1, 0, \dots, 0)^T$ 为 $n-k+1$ 维单位向量, $A^{(0)} = A = [a_1^{(0)}, a_2^{(0)}, \dots, a_n^{(0)}]$, 其中

$$a_j^{(0)} = (a_{1j}, a_{2j}, \dots, a_{nj})^T (1 \leq j \leq n)$$

记 $b_1 = a_1^{(0)}$, 不妨设 $b_1 \neq 0$, 故可取 Household 矩阵 $H_1 = I_n - \beta_1^{-1} (\tilde{b}_1 + \alpha_1 e_1)(\tilde{b}_1 + \alpha_1 e_1)^T$, 使得

$$H_1 b_1 = (\eta_1 \alpha_1, 0, \dots, 0)^T$$

其中

$$\eta_1 = \max_{1 \leq i \leq n} |a_{i1}|, \tilde{b}_1 = \frac{b_1}{\eta_1}, \alpha_1 = \|\tilde{b}_1\|_2 \text{sgn}(a_{11}), \beta_1 = \alpha_1 \left(\alpha_1 + \frac{a_{11}}{\eta_1} \right)$$

从而

$$H_1 A^{(0)} = \begin{bmatrix} -\eta_1 \alpha_1 & a_{12}^{(1)} & \dots & a_{1n}^{(1)} \\ 0 & a_{22}^{(1)} & \dots & a_{2n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2}^{(1)} & \dots & a_{nn}^{(1)} \end{bmatrix} \equiv [a_1^{(1)}, a_2^{(1)}, \dots, a_n^{(1)}] \equiv A^{(1)}.$$

如此下去, 最后得到正交矩阵 $Q = H_1 H_2 \dots H_{n-1}$, 使得 $Q^T A = R$ 为上三角矩阵, 从而 $A = QR$, 即 A 可分解成一个正交矩阵与一个上三角矩阵的乘积

2.3 Schmit 正交化方法

设 A 为 n 阶非奇异实矩阵, 即 $A = [a_1, a_2, \dots, a_n]$, 取 $b_1 = a_1 / \|a_1\|$

$$b_2' = a_2 - \langle a_2, b_1 \rangle b_1 = a_2 - \frac{\langle a_2, a_1 \rangle}{\|a_1\|} = a_2 - \frac{\langle a_2, a_1 \rangle}{\langle a_1, a_1 \rangle} a_1$$

$$\langle b_1, b_2' \rangle = 0$$

取

$$b_2 = b_2' / \|b_2'\|$$

一般地, 取 $b_k' = a_k - \sum_{i=1}^{k-1} \langle a_k, b_i \rangle b_i$, $b_k = b_k' / \|b_k'\|$

则向量组 $b_1, b_2, \dots, b_n$ 正交, 且 $\|b_k\| = 1$

即 $b_1, b_2, \dots, b_n$ 是一个正交规范向量组, 从上式中一次解出 a_k 得到

$$\begin{cases} a_1 = \|a_1\| b_1 \\ a_2 = \langle a_2, b_1 \rangle b_1 + \|b_2'\| b_2 \\ a_3 = \langle a_3, b_1 \rangle b_1 + \langle a_3, b_2 \rangle b_2 + \|b_3'\| b_3 \\ \vdots \\ a_n = \langle a_n, b_1 \rangle b_1 + \langle a_n, b_2 \rangle b_2 + \dots + \langle a_n, b_{n-1} \rangle b_{n-1} + \|b_n'\| b_n \end{cases}$$

于是 $A = [a_1, a_2, \dots, a_n]$

$$= [b_1, b_2, \dots, b_n] \begin{bmatrix} \|a_1\| & \langle a_2, b_1 \rangle & \dots & \langle a_n, b_1 \rangle \\ & \|b_2'\| & \dots & \langle a_n, b_2 \rangle \\ & & \ddots & \vdots \\ & & & \|b_{n-1}'\| & \langle a_n, b_{n-1} \rangle \\ & & & & \|b_n'\| \end{bmatrix}$$

这就是 Schmit 正交化方法对矩阵进行 QR 分解

2.4 基于平面旋转转换

设 A 为 n 阶非奇异实方阵, 则存在正交矩阵 $P_1, P_2, \dots, P_{n-1}$, 使得

$$P_{n-1}, \dots, P_2 P_1 A = R \quad (\text{上三角矩阵})$$

从而 A 有 QR 分解

$$A = QR$$

其中 P_k 为若干个平面旋转矩阵的乘积矩阵, Q 为正交矩阵 $(P_{n-1} \dots P_2 P_1)^T$ 。当 R 的主对角元素都为正时, 分解是唯一的

设 A 为 n 阶实数方阵, 对其作 QR 分解

$$A_1 = A = Q_1 R_1$$

其中 Q_1 为正交矩阵, R_1 为上三角矩阵, 交换 Q_1, R_1 的相乘顺序, 构造矩阵

$$A_2 = R_1 Q_1 = Q_1^T A_1 Q_1$$

并对齐作 QR 分解，得 $A_2 = Q_2 R_2$ ，再构造矩阵

$$A_3 = R_2 Q_2 = Q_2^T A_2 Q_2$$

一般的，按递推公式

$$\begin{cases} A_k = Q_k R_k \\ A_{k+1} = R_k Q_k = Q_k^T A_k Q_k \end{cases} \quad k = 1, 2, \dots$$

形成矩阵序列 $\{A_k\}$

QR 算法就是指以上构造矩阵序列 $\{A_k\}$ 的方法，只要 A 非奇异， $\{A_k\}$ 就可完全确定

对于任意 $A \in \mathbb{R}^{n \times n}$ ，由 QR 算法产生的序列 $\{A^{(k)}\}$ 具有以下性质：

1) $A^{(k)}$ 与 A 正交相似，从而 $A^{(k)}$ 与 A 有相同的特征值

2) 若记

$$\widetilde{Q} = Q_1 Q_2 \cdots Q_k, \quad \widetilde{R} = R_k \cdots R_2 R_1$$

则 A^k 有 QR 分解式： $A^k = \widetilde{Q} \widetilde{R}$

2.5 收敛性

由 QR 算法产生的 $\{A^{(k)}\}$ ，若当 $k \rightarrow \infty$ 时， $A^{(k)}$ 收敛于分块上三角矩阵（对角块为一阶或二阶子块），则 QR 算法是收敛的

若 $A^{(k)}$ 趋于分块上三角形状，其对角块为一阶或二阶子块，即 $A^{(k)}$ 的对角线下方元素趋于 0，则称算法是本质或基本收敛

设 $A \in \mathbb{R}^{n \times n}$ 的特征值 λ_i 满足条件 $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n| > 0$ ， $X \in \mathbb{R}^{n \times n}$ 是以 A 的特征向量为列组成的矩阵，且 $Y = X^{-1}$ 有 LU 分解，则 $\{A^{(k)}\}$ 本质或基本收敛与上三角矩阵：

$$A^{(k)} \rightarrow \begin{pmatrix} \lambda_1 & * & * & * \\ & \lambda_2 & * & * \\ & & \ddots & \vdots \\ & & & \lambda_n \end{pmatrix} \quad (k \rightarrow \infty)$$

2.5.1 带原点位移的 QR 方法

基本 QR 算法与乘幂法一样是线性收敛的，为加快其收敛速度，可使用原点位移方法进行加速。这种方法称为带原点位移的 QR 算法

设序列 $\{S_k\}$ 作

$$A^{(k)} - S_k I = Q_k R_k, A^{(k+1)} = R_k Q_k + S_k I$$

$$A^{(k+1)} = Q_k^T A^{(k)} Q_k$$

$$A^{(k+1)} = \widetilde{Q}_k^T A \widetilde{Q}_k$$

$$A^{(k+1)} - S_{k+1} I = \widetilde{Q}_k^T (A - S_{k+1} I) \widetilde{Q}_k$$

则可得:

$$\prod_{i=1}^k (A - S_i I) = \widetilde{Q}_k \widetilde{R}_k$$

令

$$\varphi_k(\lambda) = \prod_{i=1}^k (\lambda - S_i)$$

则

$$\varphi_k(A) = \prod_{i=1}^k (A - S_i I)$$

设 $A = X \Lambda X^{-1} \in \mathbb{R}^{n \times n}$, $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$, $\{S_k\}$ 是给定的序列, $\varphi_k(\lambda) = \prod_{i=1}^k (\lambda - s_i)$, 如果: $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n| > 0$; 对充分大的 k , $|\varphi_k(\lambda_i)| \neq |\varphi_k(\lambda_j)| \neq 0 (i \neq j)$; $X^{-1} = Y$ 有 LU 分解, 则 $A^{(k)} - S_k I = Q_k R_k$, $A^{(k+1)} = R_k Q_k + S_k I$ 本质收敛, 即 $A^{(k)}$ 趋于一上三角矩阵

设 A 正交相似于 Hessenberg 矩阵 H: $H = Q^T A Q$

并经由 QR 方法已算出 H 的特征值 $\lambda_i (i = 1, 2, \dots, n)$, 则可先用反乘幂法算出 H 的相应特征向量 y_i , 再根据

$$\lambda_i y_i = H y_i = Q^T A Q y_i$$

或

$$A Q y_i = \lambda_i Q y_i$$

而得到 A 关于 λ_i 的特征向量 $x_i = Q y_i (i = 1, 2, \dots, n)$ 。

实验过程及内容:

1. 问题描述 (描述需要解决的具体数值计算问题)
2. 实验步骤:
 - (1) 算法实现: 伪代码或算法框图
 - (2) 测试验证: 使用简单算例验证程序正确性
 - (3) 参数设置: 设置合适的初始值、精度要求等参数
 - (4) 运行计算: 对目标问题进行求解
 - (5) 结果记录: 记录迭代过程、最终结果和性能指标
3. 程序设计 (具体的 MATLAB 程序)

1. 问题描述

用 Jacobi 法、QR 方法和 eig 函数计算 10 阶对称三对角矩阵 A 的所有特征值, 其中 Jacobi 法和 eig 函数需要求出对应的全部特征向量, 保留五位有效数字。

$$A = \begin{bmatrix} 4 & 1 & & & & & & & & \\ 1 & 4 & 1 & & & & & & & \\ & 1 & 4 & 1 & & & & & & \\ & & 1 & 4 & 1 & & & & & \\ & & & 1 & 4 & 1 & & & & \\ & & & & 1 & 4 & 1 & & & \\ & & & & & 1 & 4 & 1 & & \\ & & & & & & 1 & 4 & 1 & \\ & & & & & & & 1 & 4 & 1 \\ & & & & & & & & 1 & 4 \end{bmatrix}$$

2. 实验步骤

2.1 Jacobi 方法

2.1.1 算法实现

伪代码如下:

```
Jacobi(A, ep)
```

1. 初始化:

```
n = size(A)
R = eye(n)    // 单位矩阵
errors = []
iter = 0
```

2. 主循环:

```
while true do
    // 查找最大非对角元
    Amax = 0
    for i = 1 to n-1
        for j = i+1 to n
            if |A[i,j]| > Amax
                Amax = |A[i,j]|
                p = i; q = j
            end if
        end for
    end for
```

```

end for

// 收敛检查
if Amax < ep then break
iter = iter + 1
errors[iter] = Amax

// 计算旋转矩阵参数
d = (A[p,p] - A[q,q]) / (2 * A[p,q])
if |d| < 1e-10
    t = 1
else
    t = sign(d) / (|d| + sqrt(1+d²))
end if
c = 1 / sqrt(1+t²)
s = c * t

// 更新矩阵 A
for k = 1 to n
    if k = p
        App = A[p,p]*c² + A[q,q]*s² + 2*A[p,q]*s*c
        Aqq = A[p,p]*s² + A[q,q]*c² - 2*A[p,q]*s*c
        Apq = (A[q,q]-A[p,p])*s*c + A[p,q]*(c²-s²)
        A[p,p] = App; A[q,q] = Aqq
        A[p,q] = Apq; A[q,p] = Apq
    else if k ≠ q
        Apk = A[p,k]*c + A[q,k]*s
        Aqk = A[p,k]*s + A[q,k]*c
        A[p,k] = Apk; A[k,p] = Apk
        A[q,k] = Aqk; A[k,q] = Aqk
    end if
end for

// 更新特征向量矩阵 R
for k = 1 to n
    Rkp = R[k,p]*c + R[k,q]*s
    Rkq = -R[k,p]*s + R[k,q]*c
    R[k,p] = Rkp; R[k,q] = Rkq
end for
end while

```

3. 输出:

```

D = diag(A)
return D, R, errors, iter

```

2.1.2 测试验证

测试矩阵:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

理论结果: 特征值 $\lambda_1 = 3, \lambda_1 = 1$, 对应的特征向量: $\omega_1 = [\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}]^T, \omega_1 = [-\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}]^T$

在 MATLAB 中输入

```
>> A = [2, 1; 1, 2];  
>> [D, R] = jacobi2(A)
```

计算出来的 D 和 R 矩阵分别为:

$$D = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}, R = \begin{bmatrix} 0.7071 & -0.7071 \\ 0.7071 & 0.7071 \end{bmatrix}$$

与理论结果相符合, 代码正确

2.1.3 参数设置

对称三对角矩阵 $A =$

$$\begin{bmatrix} 4 & 1 & & & & & & & & \\ & 4 & 1 & & & & & & & \\ & & 4 & 1 & & & & & & \\ & & & 4 & 1 & & & & & \\ & & & & 4 & 1 & & & & \\ & & & & & 4 & 1 & & & \\ & & & & & & 4 & 1 & & \\ & & & & & & & 4 & 1 & \\ & & & & & & & & 4 & 1 \\ & & & & & & & & & 4 \end{bmatrix}$$

初始值: R 为单位矩阵
误差 $\epsilon = 1e-5$

2.1.4 运行计算

在 MATLAB 中输出:

```
>> n = 10;  
a = 4; % 主对角线元素  
b = 1; % 上/下对角线元素  
A = diag(a*ones(n,1)) + diag(b*ones(n-1,1), 1) + diag(b*ones(n-1,1), -1);  
>> [D, R, errors, iter_count] = jacobi2_with_log(A)
```

2.1.5 结果记录

迭代过程 (取部分) 如下表格:

iter =							
1							
当前迭代次数: 1							
A(1) =							
4.00000	1.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000

iter =						
121						
当前迭代次数: 121						
A(121) =						
5.50108	0.00000	-0.00000	0.00000	0.00000	0.00000	0.00000
-0.00000	-0.00000	-0.00000				
0.00000	3.52115	-0.00000	-0.00000	0.00000	-0.00000	0.00000
-0.00001	0.00000	-0.00000				
-0.00000	-0.00000	4.81988	-0.00001	0.00000	-0.00000	0.00000
0.00000	0.00000	0.00000				
0.00000	-0.00000	-0.00001	2.33025	-0.00000	0.00000	0.00000
0.00000	0.00000	0.00001				
0.00000	0.00000	0.00000	-0.00000	6.37425	-0.00000	0.00000
0.00000	-0.00001	-0.00001				
0.00000	-0.00000	-0.00000	0.00000	-0.00000	3.08376	0.00000
-0.00000	0.00000	-0.00001				
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	4.51614
-0.00001	0.00000	-0.00000				
-0.00000	-0.00001	0.00000	0.00000	0.00000	-0.00000	-0.00001
1.60147	-0.00000	0.00000				
-0.00000	0.00000	0.00000	0.00000	-0.00001	0.00000	0.00000
-0.00000	5.08711	-0.00000				
-0.00000	-0.00000	0.00000	0.00001	-0.00001	-0.00001	-0.00000
0.00000	-0.00000	3.16492				
R(121) =						
0.34367	-0.11747	-0.56972	0.57503	0.21378	0.34840	0.04969
-0.19029	-0.06346	-0.05529				
0.42591	0.27463	-0.47342	-0.53111	0.29263	-0.33319	0.04944
0.15689	-0.09545	0.06443				
0.23966	-0.56485	0.13398	-0.33002	0.10154	0.51728	0.01097
0.45641	-0.08109	0.06126				
0.55967	0.06922	0.64091	0.18000	0.38863	-0.14625	-0.04190
-0.21399	-0.13333	0.03274				
-0.33372	-0.11031	-0.04751	0.31538	0.45535	-0.30748	-0.40379
0.49203	-0.26018	-0.00495				
-0.30950	0.24179	0.00887	-0.33478	0.38164	0.46267	-0.31431
-0.40760	-0.27127	-0.18804				
-0.13183	-0.65121	-0.07938	-0.16000	0.19440	-0.38490	0.22018
-0.39634	-0.02341	-0.36687				
-0.24286	0.22129	0.09648	0.05770	0.50273	0.14466	0.63413
0.20658	0.38739	-0.10427				
0.19221	0.03157	-0.00048	-0.02692	0.01174	0.01164	-0.46904
0.10422	0.64989	-0.55483				
-0.09671	-0.20556	-0.06805	-0.06888	0.25232	-0.02881	-0.24651

-0.25734	0.49825	0.70660				

iter =						
122						
当前迭代次数: 122						
A(122) =						
5.50108	0.00000	-0.00000	0.00000	0.00000	0.00000	0.00000
-0.00000	-0.00000	-0.00000				
0.00000	3.52115	-0.00000	-0.00000	0.00000	-0.00000	0.00000
-0.00001	0.00000	-0.00000				
-0.00000	-0.00000	4.81988	-0.00001	0.00000	-0.00000	0.00000
0.00000	0.00000	0.00000				
0.00000	-0.00000	-0.00001	2.33025	-0.00000	0.00000	0.00000
0.00000	0.00000	0.00001				
0.00000	0.00000	0.00000	-0.00000	6.37425	-0.00000	0.00000
0.00000	-0.00001	-0.00000				
0.00000	-0.00000	-0.00000	0.00000	-0.00000	3.08376	0.00000
-0.00000	0.00000	-0.00001				
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	4.51614
-0.00001	0.00000	-0.00000				
-0.00000	-0.00001	0.00000	0.00000	0.00000	-0.00000	-0.00001
1.60147	-0.00000	0.00000				
-0.00000	0.00000	0.00000	0.00000	-0.00001	0.00000	0.00000
-0.00000	5.08711	-0.00000				
-0.00000	-0.00000	0.00000	0.00001	-0.00000	-0.00001	-0.00000
0.00000	-0.00000	3.16492				
R(122) =						
0.34367	-0.11747	-0.56972	0.57503	0.21378	0.34840	0.04969
-0.19029	-0.06346	-0.05529				
0.42591	0.27463	-0.47342	-0.53111	0.29263	-0.33319	0.04944
0.15689	-0.09545	0.06443				
0.23966	-0.56485	0.13398	-0.33002	0.10154	0.51728	0.01097
0.45641	-0.08109	0.06126				
0.55967	0.06922	0.64091	0.18000	0.38863	-0.14625	-0.04190
-0.21399	-0.13333	0.03274				
-0.33372	-0.11031	-0.04751	0.31538	0.45535	-0.30748	-0.40379
0.49203	-0.26018	-0.00494				
-0.30950	0.24179	0.00887	-0.33478	0.38164	0.46267	-0.31431
-0.40760	-0.27127	-0.18803				
-0.13183	-0.65121	-0.07938	-0.16000	0.19440	-0.38490	0.22018

```

-0.39634  -0.02341  -0.36686
  -0.24286   0.22129   0.09648   0.05770   0.50273   0.14466   0.63413
0.20658   0.38739  -0.10427
   0.19221   0.03157  -0.00048  -0.02692   0.01174   0.01164  -0.46904
0.10422   0.64989  -0.55483
  -0.09671  -0.20556  -0.06805  -0.06888   0.25231  -0.02881  -0.24651
-0.25734   0.49825   0.70660
-----

```

最终结果: 迭代次数 $k = 122$

最终的 D 和 R 矩阵为:

$R(122)$

```

= [ 0.34367  -0.11747  -0.56972  0.57503  0.21378  0.34840  0.04969  -0.19029  -0.06346  -0.05529
    0.42591  0.27463  -0.47342  -0.53111  0.29263  -0.33319  0.04944  0.15689  -0.09545  0.06443
    0.23966  -0.56485  0.13398  -0.33002  0.10154  0.51728  0.01097  0.45641  -0.08109  0.06126
    0.55967  0.06922  0.64091  0.18000  0.38863  -0.14625  -0.04190  -0.21399  -0.13333  0.03274
    -0.33372  -0.11031  -0.04751  0.31538  0.45535  -0.30748  -0.40379  0.49203  -0.26018  -0.00494
    -0.30950  0.24179  0.00887  -0.33478  0.38164  0.46267  -0.31431  -0.40760  -0.27127  -0.18803
    -0.13183  -0.65121  -0.07938  -0.16000  0.19440  -0.38490  0.22018  -0.39634  -0.02341  -0.36686
    -0.24286  0.22129  0.09648  0.05770  0.50273  0.14466  0.63413  0.20658  0.38739  -0.10427
    0.19221  0.03157  -0.00048  -0.02692  0.01174  0.01164  -0.46904  0.10422  0.64989  -0.55483
    -0.09671  -0.20556  -0.06805  -0.06888  0.25231  -0.02881  -0.24651  -0.25734  0.49825  0.70660 ]

```

```

D = [ 5.5011  0  0  0  0  0  0  0  0  0
      0  3.5212  0  0  0  0  0  0  0  0
      0  0  4.8199  0  0  0  0  0  0  0
      0  0  0  2.3303  0  0  0  0  0  0
      0  0  0  0  6.3742  0  0  0  0  0
      0  0  0  0  0  3.0838  0  0  0  0
      0  0  0  0  0  0  4.5161  0  0  0
      0  0  0  0  0  0  0  1.6015  0  0
      0  0  0  0  0  0  0  0  5.0871  0
      0  0  0  0  0  0  0  0  0  3.1649 ]

```

故特征值为:

$\lambda = [5.5011, 3.5212, 4.8199, 2.3303, 6.3742, 3.0838, 4.5161, 1.6015, 5.0871, 3.1649]^T$

2.2 QR 方法

2.2.1 算法实现

伪代码实现

```

function QR_ALGORITHM(A, max_iter, tol)
    // 1. 参数初始化
    if max_iter 未提供 then max_iter ← 500
    if tol 未提供 then tol ← 1e-5
    n ← 矩阵 A 的行数
    Ak ← A                                // 当前迭代矩阵
    Q_all ← n×n 单位矩阵                 // 累积的正交变换矩阵
    converged ← false                     // 收敛标志

    // 2. QR 主迭代循环
    for iter_count = 1 to max_iter do
        // 2.1 QR 分解
        (Q, R) ← QR_DECOMPOSITION(Ak)    // Ak = Q * R

        // 2.2 更新累积正交矩阵

```

```

Q_all ← Q_all * Q

// 2.3 更新迭代矩阵
Ak ← R * Q

// 2.4 收敛性检查
max_lower ← 0
for i = 2 to n do
    for j = 1 to i-1 do
        if |Ak[i, j]| > max_lower then
            max_lower ← |Ak[i, j]|
        end if
    end for
end for

// 2.5 判断收敛
if max_lower < tol then
    converged ← true
    输出 "在第" iter_count "次迭代收敛到上三角矩阵"
    break
end if
end for

// 3. 处理收敛状态
if not converged then
    输出 "达到最大迭代次数但未完全收敛"
    输出 "下三角最大元素:" max_lower
end if

// 4. 提取特征值
eigenvalues ← 对角线元素(Ak) // eigenvalues = diag(Ak)
H ← Ak // 保存最终的上三角矩阵 H

// 5. 初始化特征向量矩阵
eigenvectors ← 零矩阵(n, n) // 用于存储 A 的特征向量
eigenvectors_H ← 零矩阵(n, n) // 用于存储 H 的特征向量

for i = 1 to n do
    lambda ← eigenvalues[i] // 当前特征值

    // 6.1 构造矩阵 (H - λI)
    B ← H - lambda * 单位矩阵(n)

    // 6.2 初始化随机向量

```

```

v ← 生成随机向量(n, 1)
v ← v / 向量范数(v)      // 归一化

// 6.3 反幂法参数设置
max_inner ← 100           // 反幂法最大迭代次数
inner_tol ← 1e-12         // 反幂法收敛容差
converged_inner ← false

// 6.4 反幂法迭代
for iter_inner = 1 to max_inner do
    v_old ← v              // 保存旧向量

    // 解线性方程组:  $(H - \lambda I) * v_{\text{new}} = v_{\text{old}}$ 
    v_new ← 线性方程组求解(B, v_old)

    // 归一化新向量
    v ← v_new / 向量范数(v_new)

    // 检查收敛
    if 向量范数(v - v_old) < inner_tol then
        converged_inner ← true
        break
    end if
end for

// 6.5 保存 H 的特征向量
y_i ← v
eigenvectors_H[:, i] ← y_i

// 6.6 计算残差 (验证)
residual ← 向量范数(H * y_i - lambda * y_i)
输出 "λ" i "=" lambda " " iter_inner "次迭代 残差:" residual

// 6.7 还原 A 的特征向量:  $x_i = Q_{\text{all}} * y_i$ 
x_i ← Q_all * y_i
eigenvectors[:, i] ← x_i
end for

return eigenvalues, eigenvectors, iter_count
end function

```

2.2.2 测试验证

测试矩阵:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

理论结果：特征值 $\lambda_1 = 3, \lambda_2 = 1$ ，对应的特征向量： $\omega_1 = [\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}]^T, \omega_2 = [-\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}]^T$

在 MATLAB 中输入

```
>> A = [2,1;1,2];
>> [m, k, a] = qr_algorithm(A)
```

计算出来的最终 A 矩阵为：

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

特征向量矩阵为：

$$R = \begin{bmatrix} 0.7071 & 0.7071 \\ 0.7071 & -0.7071 \end{bmatrix}$$

最终的特征值： $\lambda = [3, 1]^T$

与理论结果相符合，代码正确

2.2.3 参数设置

对称三对角矩阵 $A =$

$$\begin{bmatrix} 4 & 1 & & & & & & & & \\ & 4 & 1 & & & & & & & \\ & & 4 & 1 & & & & & & \\ & & & 4 & 1 & & & & & \\ & & & & 4 & 1 & & & & \\ & & & & & 4 & 1 & & & \\ & & & & & & 4 & 1 & & \\ & & & & & & & 4 & 1 & \\ & & & & & & & & 4 & 1 \\ & & & & & & & & & 4 \end{bmatrix}$$

误差 $\epsilon = 1e-5$

2.2.4 运行计算

在 MATLAB 中输入

```
>> n = 10;
a = 4; % 主对角线元素
b = 1; % 上/下对角线元素
A = diag(a*ones(n,1)) + diag(b*ones(n-1,1), 1) + diag(b*ones(n-1,1), -1);
>> [m, v, errors, iter_count] = jacobi2_with_log(A)
```

2.2.5 实验结果

迭代过程如下：

```
迭代次数: 1
矩阵 A(1):
4.00000 1.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000
1.00000 4.00000 1.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000
```

0.00000	1.00000	4.00000	1.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
0.00000	0.00000	1.00000	4.00000	1.00000	0.00000	0.00000	0.00000	0.00000	0.00000
0.00000	0.00000	0.00000	1.00000	4.00000	1.00000	0.00000	0.00000	0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	1.00000	4.00000	1.00000	0.00000	0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	4.00000	1.00000	0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	4.00000	1.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	4.00000	1.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	4.00000

迭代次数: 2

矩阵 A(2):

4.47059	0.91508	0.00000	0.00000	0.00000	-0.00000	0.00000	-0.00000	0.00000	0.00000
0.91508	4.05834	0.99024	0.00000	0.00000	-0.00000	0.00000	-0.00000	0.00000	0.00000
0.00000	0.99024	4.00630	0.99901	-0.00000	0.00000	-0.00000	0.00000	-0.00000	-0.00000
0.00000	0.00000	0.99901	4.00061	0.99991	0.00000	-0.00000	-0.00000	-0.00000	0.00000
0.00000	0.00000	0.00000	0.99991	4.00006	0.99999	0.00000	-0.00000	0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.99999	4.00000	1.00000	0.00000	-0.00000	-0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	4.00000	1.00000	0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	4.00000	1.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	4.00000	-0.96343
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	-0.96343	3.46410

.....

迭代次数: 262

矩阵 A(262):

5.91899	0.00001	0.00000	-0.00000	-0.00000	-0.00000	0.00000	-0.00000	0.00000	0.00000
0.00001	5.68251	0.00000	0.00000	-0.00000	-0.00000	0.00000	-0.00000	-0.00000	-0.00000
0.00000	0.00000	5.30972	0.00000	0.00000	-0.00000	0.00000	-0.00000	-0.00000	-0.00000
0.00000	0.00000	0.00000	4.83083	0.00000	-0.00000	-0.00000	0.00000	-0.00000	-0.00000
0.00000	0.00000	0.00000	0.00000	4.28463	0.00000	-0.00000	-0.00000	0.00000	-0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	3.71537	0.00000	-0.00000	0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	3.16917	0.00000	-0.00000	-0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	2.69028	-0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	2.31749	-0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	-0.00000	2.08101

迭代次数: 263

矩阵 A(263):

5.91899	0.00001	0.00000	-0.00000	-0.00000	-0.00000	0.00000	-0.00000	0.00000	-0.00000
0.00001	5.68251	0.00000	0.00000	-0.00000	-0.00000	0.00000	-0.00000	-0.00000	0.00000
0.00000	0.00000	5.30972	0.00000	0.00000	-0.00000	0.00000	-0.00000	-0.00000	0.00000
0.00000	0.00000	0.00000	4.83083	0.00000	-0.00000	-0.00000	0.00000	-0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	4.28463	0.00000	-0.00000	-0.00000	0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	3.71537	0.00000	-0.00000	0.00000	-0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	3.16917	0.00000	-0.00000	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	2.69028	-0.00000	-0.00000

0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	2.31749	0.00000
0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	2.08101

最终结果:

迭代次数: k=263

最终的 A_k 矩阵:

$$A_k = \begin{bmatrix} 5.91899 & 0.00001 & 0.00000 & -0.00000 & -0.00000 & -0.00000 & 0.00000 & -0.00000 & 0.00000 & 0.00000 \\ 0.00001 & 5.68251 & 0.00000 & 0.00000 & -0.00000 & -0.00000 & 0.00000 & -0.00000 & -0.00000 & 0.00000 \\ 0.00000 & 0.00000 & 5.30972 & 0.00000 & 0.00000 & -0.00000 & 0.00000 & -0.00000 & -0.00000 & 0.00000 \\ 0.00000 & 0.00000 & 0.00000 & 4.83083 & 0.00000 & -0.00000 & -0.00000 & 0.00000 & -0.00000 & -0.00000 \\ 0.00000 & 0.00000 & 0.00000 & 0.00000 & 4.28463 & 0.00000 & -0.00000 & -0.00000 & 0.00000 & -0.00000 \\ 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 3.71537 & 0.00000 & -0.00000 & 0.00000 & 0.00000 \\ 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 3.16917 & 0.00000 & -0.00000 & -0.00000 \\ 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 2.69028 & -0.00000 & 0.00000 \\ 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 2.31749 & -0.00000 \\ 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 & 2.08101 \end{bmatrix}$$

最终的特征向量:

$$v = \begin{bmatrix} -0.12013 & -0.23053 & 0.32225 & -0.38787 & -0.42206 & -0.42206 & -0.38787 & -0.32225 & 0.23053 & 0.12013 \\ -0.23053 & -0.38787 & 0.42206 & -0.32225 & -0.12013 & 0.12013 & 0.32225 & 0.42206 & -0.38787 & -0.23053 \\ -0.32225 & -0.42206 & 0.23053 & 0.12013 & 0.38787 & 0.38787 & 0.12013 & -0.23053 & 0.42206 & 0.32225 \\ -0.38787 & -0.32225 & -0.12013 & 0.42206 & 0.23053 & -0.23053 & -0.42206 & -0.12013 & -0.32225 & -0.38787 \\ -0.42206 & -0.12013 & -0.38787 & 0.23053 & -0.32225 & -0.32225 & 0.23053 & 0.38787 & 0.12013 & 0.42206 \\ -0.42206 & 0.12013 & -0.38787 & -0.23053 & -0.32225 & 0.32225 & 0.23053 & -0.38787 & 0.12013 & -0.42206 \\ -0.38787 & 0.32225 & -0.12013 & -0.42206 & 0.23053 & 0.23053 & -0.42206 & 0.12013 & -0.32225 & 0.38787 \\ -0.32225 & 0.42206 & 0.23053 & -0.12013 & 0.38787 & -0.38787 & 0.12013 & 0.23053 & 0.42206 & -0.32225 \\ -0.23053 & 0.38787 & 0.42206 & 0.32225 & -0.12013 & -0.12013 & 0.32225 & -0.42206 & -0.38787 & 0.23053 \\ -0.12013 & 0.23053 & 0.32225 & 0.38787 & -0.42206 & 0.42206 & -0.38787 & 0.32225 & 0.23053 & -0.12013 \end{bmatrix}$$

矩阵 A 的特征值为:

$$\lambda = [5.9190, 5.6825, 5.3097, 4.8308, 4.2846, 3.7154, 3.1692, 2.6902, 2.3175, 2.0810]^T$$

2.3 MATLAB 中 eig 函数

在 MATLAB 中输入:

```
>> n = 10;
a = 4; % 主对角线元素
b = 1; % 上/下对角线元素
A = diag(a*ones(n,1)) + diag(b*ones(n-1,1), 1) + diag(b*ones(n-1,1), -1);
>> [V, D] = eig(A);
```

其中 V 为特征向量矩阵, D 为对角矩阵, 其中对角线为 A 的特征值

计算结果:

$$V = \begin{bmatrix} -0.1201 & -0.2305 & -0.3223 & -0.3879 & 0.4221 & 0.4221 & 0.3879 & 0.3223 & -0.2305 & 0.1201 \\ 0.2305 & 0.3879 & 0.4221 & 0.3223 & -0.1201 & 0.1201 & 0.3223 & 0.4221 & -0.3879 & 0.2305 \\ -0.3223 & -0.4221 & -0.2305 & 0.1201 & -0.3879 & -0.3879 & -0.1201 & 0.2305 & -0.4221 & 0.3223 \\ 0.3879 & 0.3223 & -0.1201 & -0.4221 & 0.2305 & -0.2305 & -0.4221 & -0.1201 & -0.3223 & 0.3879 \\ -0.4221 & -0.1201 & 0.3879 & 0.2305 & 0.3223 & 0.3223 & -0.2305 & -0.3879 & -0.1201 & 0.4221 \\ 0.4221 & -0.1201 & -0.3879 & 0.2305 & -0.3223 & 0.3223 & 0.2305 & -0.3879 & 0.1201 & 0.4221 \\ -0.3879 & 0.3223 & 0.1201 & -0.4221 & -0.2305 & -0.2305 & 0.4221 & -0.1201 & 0.3223 & 0.3879 \\ 0.3223 & -0.4221 & 0.2305 & 0.1201 & 0.3879 & -0.3879 & 0.1201 & 0.2305 & 0.4221 & 0.3223 \\ -0.2305 & 0.3879 & -0.4221 & 0.3223 & 0.1201 & -0.1201 & -0.3223 & 0.4221 & 0.3879 & 0.2305 \\ 0.1201 & -0.2305 & 0.3223 & -0.3879 & -0.4221 & 0.4221 & -0.3879 & 0.3223 & 0.2305 & -0.1201 \end{bmatrix}$$

$$D = \begin{bmatrix} 2.0810 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2.3175 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2.6903 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3.1692 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3.7154 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4.2846 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 4.8308 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 5.3097 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 5.6825 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 5.9190 \end{bmatrix}$$

特征值为:

$$\begin{aligned}\lambda_1 &= 2.0810, & \lambda_2 &= 2.3175, & \lambda_3 &= 2.6903, \\ \lambda_4 &= 3.1692, & \lambda_5 &= 3.7154, & \lambda_6 &= 4.2846, \\ \lambda_7 &= 4.8308, & \lambda_8 &= 5.3097, & \lambda_9 &= 5.6825, \\ \lambda_{10} &= 5.9190\end{aligned}$$

3. 程序设计

3.1 Jacobi 方法

```
function [D, R] = jacobi2(A, ep)
if nargin < 2 ep = 1e-5; end
n = length(A); R = eye(n);
while 1
    Amax = 0;
    for l = 1 : n - 1
        for k = l + 1 : n
            if abs(A(l, k)) > Amax
                Amax = abs(A(l, k))
                i = l; j = k;
            end
        end
    end
    if Amax < ep break; end
    % 计算三角函数
    d = (A(i, i) - A(j, j)) / (2 * A(i, j));
    if abs(d) < 1e-10
        t = 1;
    else
        t = sign(d) / (abs(d) + sqrt(d^2 + 1));
    end
    c = 1 / sqrt(t^2 + 1); s = c * t;
    % 旋转计算
    for l = 1 : n
        if l == i
            Aii = A(i, i) * c^2 + A(j, j) * s^2 + 2 * A(i, j) * s * c;
            Ajj = A(i, i) * s^2 + A(j, j) * c^2 - 2 * A(i, j) * s * c;
            A(i, j) = (A(j, j) - A(i, i)) * s * c + A(i, j) * (c^2 - s^2);
            A(j, i) = A(i, j);
            A(i, i) = Aii;
            A(j, j) = Ajj;
        elseif l == j
            Ail = A(i, l) * c + A(j, l) * s;
            Ajl = A(i, l) * s + A(j, l) * c;
            A(i, l) = Ail; A(l, i) = Ail;
            A(j, l) = Ajl; A(l, j) = Ajl;
        end
    end
end
```

```

        Rli = R(l, i) * c + R(l, j) * s;
        Rlj = -R(l, i) * s + R(l, j) * c;
        R(l, i) = Rli; R(l, j) = Rlj
    end
end
D = diag(diag(A))
end

```

3.2 QR 方法

```

function [eigenvalues, iter_count, Ak_final] = qr_algorithm(A, max_iter, tol)
    if nargin < 3
        tol = 1e-5;
    end
    if nargin < 2
        max_iter = 500;
    end

    n = size(A, 1);
    Q_all = eye(n);
    Ak = A;
    converged = false;
    for iter_count = 1:max_iter
        [Q, R] = qr(Ak);
        Ak = R * Q;
        Q_all = Q_all * Q;
        max_lower = 0;
        for i = 2:n
            for j = 1:i-1
                if abs(Ak(i, j)) > max_lower
                    max_lower = abs(Ak(i, j));
                end
            end
        end
        if max_lower < tol
            converged = true;
            break;
        end
    end

    if ~converged
        fprintf('\n  达到最大迭代次数 %d 但未完全收敛\n', max_iter);
        fprintf('  下三角最大元素: %.6e\n', max_lower);
    end
end

```

```

eigenvalues = diag(Ak);
Ak_final = Ak;
eigenvalues = diag(Ak);
eigenvectors = zeros(n,n);
eigenvectors_H = zeros(n, n); % H 的特征向量
H = Ak;
for i = 1:n
    lambda = eigenvalues(i);
    % 1. 构造 (H - λI)
    B = H - lambda * eye(n);
    % 2. 初始向量 (随机生成, 归一化)
    v = randn(n, 1);
    v = v / norm(v);

    % 3. 反幂法迭代参数
    max_inner = 100; % 反幂法最大迭代次数
    inner_tol = 1e-12; % 反幂法收敛容差
    converged = false;
    % 4. 反幂法迭代
    for iter_inner = 1:max_inner
        % 保存旧向量
        v_old = v;
        % 解方程: (H - λI) * v_new = v_old
        v_new = B \ v_old;
        % 归一化
        v = v_new / norm(v_new);
        % 检查收敛: 向量变化很小
        if norm(v - v_old) < inner_tol
            converged = true;
            break;
        end
    end
end

% 5. 结果处理
y_i = v; % H 的特征向量
eigenvectors_H(:, i) = y_i;
% 6. 还原 A 的特征向量: x_i = Q * y_i
x_i = Q_all * y_i;
eigenvectors(:, i) = x_i;
end
end

```

实验结果与分析:

1. 实验结果展示（采用列表形式展示计算结果，表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等，并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等）

方法	计算时间	迭代次数
Jacobi 方法	0.055502s	k = 122
QR 方法	0.457907s	k = 262

最终结果（最终的特征值）：

Jacobi 方法:

$$\lambda = [5.5011, 3.5212, 4.8199, 2.3303, 6.3742, 3.0838, 4.5161, 1.6015, 5.0871, 3.1649]^T$$

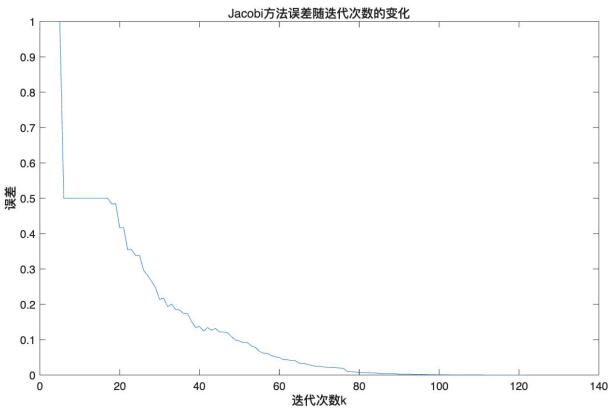
QR 方法:

$$\lambda = [5.9190, 5.6825, 5.3097, 4.8308, 4.2846, 3.7154, 3.1692, 2.6902, 2.3175, 2.0810]^T$$

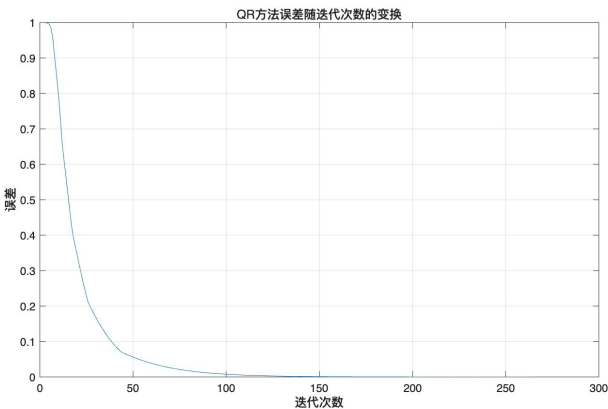
MATLAB 中 eig 函数计算结果:

$$\lambda = [5.9190, 5.6825, 5.3097, 4.8308, 4.2846, 3.7154, 3.1692, 2.6903, 2.3175, 2.0810]^T$$

误差随迭代次数的变化曲线图:



图一 Jacobi 方法误差随迭代次数变化曲线图



图二 QR 方法误差随迭代次数变化曲线图

2. 结果分析与讨论思考

(1) 计算结果分析与比较方法的优缺点及其适用场景等

可以看出，Jacobi 方法迭代速度少，但 QR 分解收敛速度较快，但最终得出的结果精度与 QR 分解得出的解的精度较差，QR 分解得出的结果更加接近于真实值

Jacobi 方法:

优点:

1. 算法简单直观
2. 特征向量直接得到 (正交)
3. 数值稳定

缺点:

1. 不适合大规模矩阵
2. 只适用于对称矩阵

适用场景:

小规模对称矩阵教学演示

QR 分解:

优点:

1. 收敛较快
2. 适用于一般方阵
3. 可处理复特征值

缺点:

1. 需额外求特征向量
2. 实现复杂
3. 需处理上 Hessenberg 化

适用范围:

中小规模一般矩阵特征值问题

(2) 上机实验中遇到问题及其解决方案

问题: QR 算法特征向量求解

现象: QR 迭代只得到上三角矩阵 H 和累积正交矩阵 Q , 特征向量需额外计算

分析: 需用反幂法对每个特征值单独求解

解决方案: 实现反幂法 $(H - \lambda_i I)y_i \approx 0$, 然后 $x_i = Q y_i$

(3) 算法改进及其建议

Jacobi 方法优化:

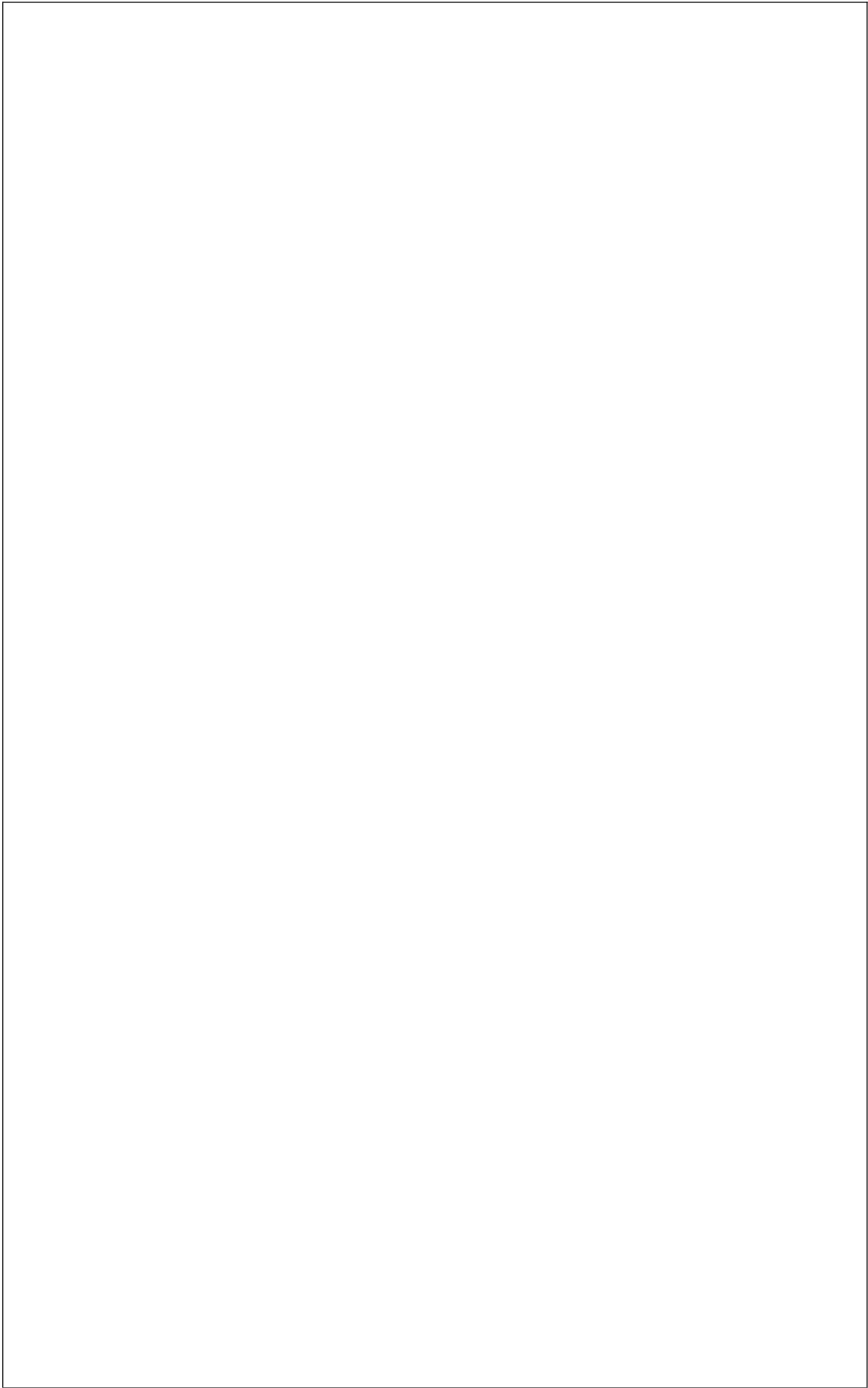
- 实现阈值 Jacobi: 只消去大于某个阈值的非对角元
- 循环 Jacobi: 按固定顺序遍历所有非对角元, 避免每次寻找最大值
- 并行化: 现代计算机可同时处理多个旋转

QR 算法加速:

- 位移策略: 引入 Wilkinson 位移加速收敛
- 隐式 QR: 避免显式计算 Q 矩阵, 减少存储和计算量
- 双重步位移: 处理复特征值对

工程实践建议:

- 小规模矩阵 ($n < 100$): 可用 Jacobi 或简单 QR
- 中等规模 ($100 < n < 1000$): 使用带位移的隐式 QR
- 大规模稀疏矩阵: 使用 Arnoldi 迭代 (eigs 函数)



实验结论:

- 1. 通过本次实验, 成功实现了 Jacobi 方法和 QR 方法的算法编程
- 2. 验证了 Jacobi 方法和 QR 方法的收敛性和计算精度
- 3. 比较了不同方法的性能特点, 加深了对数值计算方法的理 解
- 4. 掌握了数值实验的基本流程和结果分析方法

指导教师批阅意见:

成绩评定:

指导教师签字:
年 月 日

备注:

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。